Each of the four agents is realized as an LLM-backed module driven by `claude-sonnet-5` at temperature 0.7, orchestrated as a LangGraph state machine through a common controller that manages prompting, tool invocation, and the per-agent execute-and-repair loop. Temperature is deliberately not zero: the protocol calls for three runs per subject with different seeds so that variance can be reported, and at temperature zero those runs would be near-identical.

Execution and sandboxing. Local verification requires running untrusted, machine-generated code. Each candidate clone is staged into an isolated temporary directory and executed with the host interpreter under a wall-clock timeout, capturing compiler diagnostics, exit status, standard output, and assertion failures. We note this as a limitation rather than a strength: the isolation is filesystem-level, not container-level, so a clone that reached the network or the wider filesystem would not be contained. The corpus is our own and the generated code is inspected, but a deployment on untrusted subjects would require a container or VM boundary.

The corpus is twelve Python programs we authored for this study, modelled on industrial measurement and control software: CRC framing, sensor serialisation, timestamp parsing, SCADA polling, Modbus register access, MQTT telemetry, PID control, a tank-level state machine, threshold alarming, and three multi-module supervisory programs. Each ships a `pytest` suite and recorded input/output traces that serve as both the *VCA*'s test corpus and the equivalence oracle's ground truth. Authoring rather than harvesting the corpus was a deliberate trade: it costs external validity, and we treat that as a threat in Section VI, but it is what makes per-program golden vectors and invariants available at all, and without those the equivalence question cannot be asked at the granularity this study needs.

Two subjects, `modbus_meter_reader` (B2) and `boiler_control_loop` (D1), declare a substitution that cannot be satisfied as stated: both originals speak Modbus TCP via `ModbusTcpClient(host, port=502)`, while the declared target `minimalmodbus` is serial-only and exposes no TCP entry point. We discovered this during evaluation rather than by design. Rather than repair the corpus we retained both as *negative controls*, and they proved informative: across the nine D1 runs, five independently chose `pyModbusTCP` over the declared target, and the single run that did adopt `minimalmodbus` was caught by the factory-execution check passing a TCP port number as a Modbus slave address.

Structural divergence is reported as a composite of two rename-insensitive measures taken between the delivered clone and the pristine original, over the concatenation of all first-party files. The first is the Ratcliff/Obershelp similarity of the pre-order AST *node-type* sequence, which ignores identifier and literal choices and so cannot be inflated by renaming. The second is a weighted Jaccard distance over a depth-tagged histogram of branch constructs, which captures reshaped control flow. The composite is their unweighted mean, on $[0, 1]$, higher meaning more divergent.

We report a rename-insensitive measure deliberately. Token-level edit distance rewards renaming every identifier, which is exactly the kind of cosmetic change a Type II clone detector defeats, and a diversification result that a de-

tector defeats is not a result.

We run *Proteus* with `claude-sonnet-5` at temperature 0.7, three seeds per subject, a wall-clock ceiling of 180 s per LLM request, and a cap of three *VCA*-triggered stage re-entries per run. Every subject receives the same requirement: remove the dependency named in its `requirements.txt` while preserving observable behaviour, and additionally, for Categories C and D, diversify all first-party logic.

Crucially, **the replacement library is never named** — not to *Proteus* and not to the baseline. Both must select a target themselves. An earlier protocol supplied the corpus's declared target to *Proteus* while the baseline chose freely; that made the comparison asymmetric and, on the two negative controls, forced *Proteus* toward a library that cannot work. Under free choice the two systems are asked the same question.

Baselines. To isolate the contribution of *Proteus*'s verification-driven design we compare against two configurations. *Baseline 1 (single-shot LLM)* is one prompt to the same model, naming only the dependency to be removed, with no agents, no tools, and no verification: a single API call whose reply is parsed for code blocks. We confirmed by inspection and by telemetry that it uses no tool loop — exactly one LLM call on all 36 runs, averaging 1,471 input tokens, against up to 14 calls and 20,749 input tokens per run for *Proteus*. *Baseline 2 (*Proteus *without verification)* is an ablation with the *VCA* and the global oracle disabled, leaving only the per-agent local execute-and-repair loop.

Table **??** reports the conventional measures. They do not separate the three configurations: generation success is 30/36, 33/36 and 29/36, and equivalence confirmation on Categories C and D is 16/18, 17/18 and 16/18. On these numbers alone the ablation is the strongest configuration and full *Proteus* the weakest, which is the opposite of what the two-tier discipline predicts.

The measures are not wrong; they are answering a narrower question than they appear to. Every subject's tests pass before migration, and every subject injects its I/O dependency, so the factory code that performs the substitution is frequently never executed by the suite. A clone can therefore compile, pass its full suite, and satisfy the oracle while calling the new library incorrectly on the one path the tests do not reach.

To measure what those instruments cannot, we submitted all 108 clones to a blind LLM review (Table **??**). Each clone was shown alongside its original to `claude-opus-5` — deliberately a different and stronger model than the `claude-sonnet-5` that generated it, so that no model grades its own output — and asked only for concrete behavioural differences, each naming a file, quoting a line, and stating the runtime consequence. Verdicts are BROKEN (fails on ordinary input), MAJOR (wrong behaviour in a case the original handled), or MINOR (a real difference in an edge case). All system-identifying markers were renamed or removed before review, and the batch aborts rather than continuing if any marker survives, so the reviewer never learns which configuration produced a clone.

The review separates the configurations where the conventional measures do not. Of the clones each configuration *itself accepted* — `migration_success`:

compiles, passes its suite, genuinely changed, and the dependency verifiably removed — the fraction carrying BROKEN or MAJOR defects is 54% for the single-shot baseline, 55% for the ablation, and 33% for full *Proteus* (Table **??**, Fig. **??**).

Two observations follow, and they point in opposite directions.

First, **the ablation performs like the single-shot baseline**: 55% against 54%. Removing the verification tier leaves an agent architecture that is no more reliable than one prompt. Whatever *Proteus* contributes is attributable to verification rather than to decomposition into agents.

Second, **full *Proteus* does not deliver more sound clones than one prompt**. It yields 10 defect-free accepted clones per 36 runs against the baseline's 12. It accepts far fewer clones (15 against 28) and is right more often when it does accept, which is a precision gain paid for in recall. On a corpus where a wrong clone is worse than no clone — boiler supervisors, pump controllers — that is the trade we would choose, but it is a trade and not a dominance result.

Cost. The precision is not free. Full *Proteus* consumed 925K tokens and 140 minutes across 36 runs against the baseline's 95K tokens and 6 minutes, roughly a sixfold increase in tokens and a twentyfold increase in wall time, for $4.91 against $0.79.

Statistical caution. With twelve subjects the differences above are not statistically significant: Fisher's exact test gives $p = 0.34$ for the single-shot baseline against full *Proteus* on defect rate, and $p = 1.00$ against the ablation. The 95% confidence intervals overlap substantially (54% [36–70], 55% [28–79], 33% [15–58]). We report the direction and its mechanism, not a demonstrated effect; separating 33% from 54% at $p < 0.05$ would require roughly three to four times this corpus.

Which stage dominates repair cost. Full *Proteus* triggered six *VCA* stage re-entries across 36 runs, all routed to the *GSA*. No run exhausted its repair budget. Repair cost is therefore not the binding constraint on this corpus; acceptance is.

Which failure classes are hardest. The blind review's findings fall into a small number of recurring shapes, and the two systems fail differently. The single-shot baseline produces 80 findings dominated by *fidelity* failures: 34 concern a changed exception type or message, and 10 concern a client or connection that is never closed. Its wiring is correct and its semantics drift. *Proteus* produces 45 findings dominated by *integration* failures: arguments accepted by an adapter and then never used (7 against the baseline's 2), adapters that are generated but not reached (5 against 2), and errors collapsed into a single generic exception (4 against 1). Its adapters are structurally sound — generating a wrapper naturally preserves lifecycle and error translation, which is why it has one unclosed resource against the baseline's ten — and it loses detail at the call sites.

Two illustrative defects. In `proteus_no_verif/D1_boiler_control_loop_run2/transport.py`, an adapter catches `(ModbusError, Exception)` and converts every failure — connection loss, and programming errors such as `TypeError` alike — into a generic `IOError`. On a boiler supervisor an adapter bug becomes indistin-

guishable from a genuine Modbus fault. The ablation accepted this clone; full *Proteus* rejected its counterpart at the *VCA* on the same two seeds. In `proteus_full/D1_boiler_control_loop_run3/transport.py`, the generated wrappers accept the per-call slave identifier and never use it, so an injected client configured for a different device would be actuated instead. Neither defect is reachable by the test suite, because the suite injects a fake client that ignores the same argument.

What the global oracle catches. Two *Proteus* clones import the migrated-to library while still calling the migrated-from one, which was never imported — a guaranteed `NameError` on the factory path. Both were rejected by the pipeline, at the *LMA* and *VCA* respectively, and the blind reviewer independently rated both BROKEN. The system's own verdict and an independent reviewer's agree on exactly the clones the test suite passes.

a) Lessons Learned: Three findings we did not anticipate.

Test suites are the wrong instrument for this task. Dependency injection is good practice and it makes the migrated code path untestable by the very suite that certifies it. Every configuration compiles at 36/36; compilation discriminates nothing. Any future benchmark of clone generation needs an oracle that executes the substituted path directly, which is why we added factory execution as a fifth layer mid-study.

Separating substitution from mutation keeps repairs local, but the seam leaks. The *LSA*/*GSA* split behaves as intended — repairs stay within one agent and no run exhausted its budget — yet the residual failure mode lives precisely at the boundary, where an adapter is generated correctly and then not fully wired to the call sites it was written for. Orthogonal concerns still require a shared contract at their interface.

One third of the corpus is unsolved by every configuration. Four subjects — the MQTT bridge, the PID controller, the threshold engine, and the telemetry gateway — produced zero defect-free accepted clones across all 27 runs. Category C is 1 of 27 across all configurations. The requirement to replace a dependency with first-party code is, on this evidence, not differentially solved but essentially unsolved: *Proteus*'s resolver selects among libraries and has no mechanism to synthesise a replacement implementation, and while the single-shot baseline does write a PID controller and a state machine by hand, only one such clone survived review.

Proteus produces working clones for 29 of 36 runs across twelve subject programs at a median of zero repair iterations, and the oracle confirms behavioural equivalence on 16 of 18 Category C and D runs. A blind review of all 108 generated clones finds that 33% of the clones *Proteus* accepts carry serious defects, against 54% for an unverified single-shot baseline — though *Proteus* accepts far fewer clones, and on this corpus size the difference is not statistically significant. These early results suggest that agentic, verification-driven mutation is a promising route to automated, trustworthy software diversification, and that the verification tier rather than the agent decomposition is what carries the effect.

On software that estimates factory measurements and controls industrial

Table 1: Clone-generation results on the industrial measurement and control corpus. Twelve programs × three seeds = 36 runs per configuration. *Equiv. conf.* is reported over categories C and D only — six subjects, 18 runs per configuration — these being the programs that ship golden vectors and invariants; categories A and B have no equivalence probes and are excluded rather than scored as passes.

Configuration	Gen. success	Equiv. conf.	Divergence	Repair iters	Cost (USD)
Single-shot LLM (B1)	30/36	16/18	0.141	0 (0 re-entry)	$0.79
Proteus w/o verification (B2)	33/36	17/18	0.127	0 (0 re-entry)	$3.87
Proteus (full)	29/36	16/18	0.154	6 (6 re-entry)	$4.91

Table 2: Blind LLM review of all 108 generated clones. An independent model (`claude-opus-5`, distinct from the `claude-sonnet-5` used for generation) was shown each original and its clone with all system-identifying markers removed, and asked for concrete behavioural defects. NO-OP denotes a clone byte-identical to its original: trivially faithful because nothing was attempted, and therefore excluded from the clean rate rather than counted as a success.

Configuration	BROKEN	MAJOR	MINOR	CLEAN	NO-OP	Clean rate
Single-shot LLM (B1)	2	16	12	5	0	5/36
Proteus w/o verification (B2)	1	8	1	15	11	15/25
Proteus (full)	5	8	4	16	3	16/33

infrastructure, *Proteus* generates equivalence-confirmed clones for 16 of 18 Category C and D runs at a median of zero repair iterations, and reduces the share of accepted clones carrying serious behavioural defects from 54% to 33% relative to an unverified single-shot baseline — a difference in the predicted direction that this corpus is too small to establish ($p = 0.34$).

Table 3: Precision of each configuration's own success signal. Of the clones a configuration certified (compiles, passes the full suite, and genuinely substituted the dependency), the fraction an independent blind reviewer found to carry BROKEN or MAJOR behavioural defects. Accepting more clones and accepting sound clones are different capabilities, and they trade off against one another.

Configuration	Clones accepted	With serious defects	Defect rate
Single-shot LLM (B1)	28/36	15	**54%**
Proteus w/o verification (B2)	11/36	6	**55%**
Proteus (full)	15/36	5	**33%**

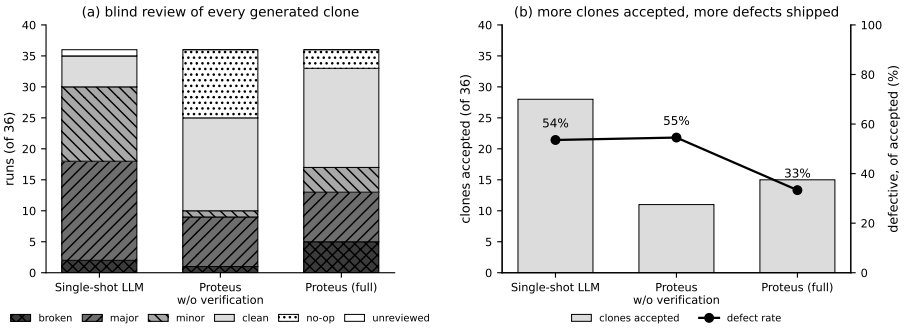


Figure 1: test